MLOps

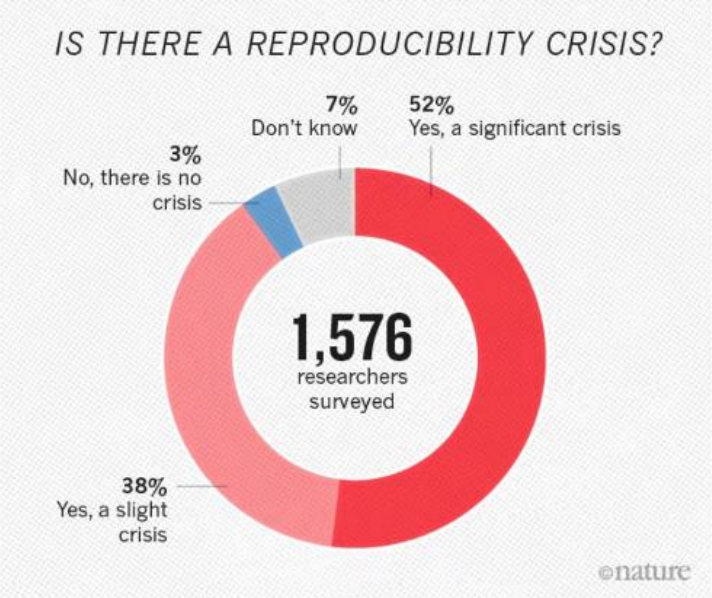
What is it and how to use it

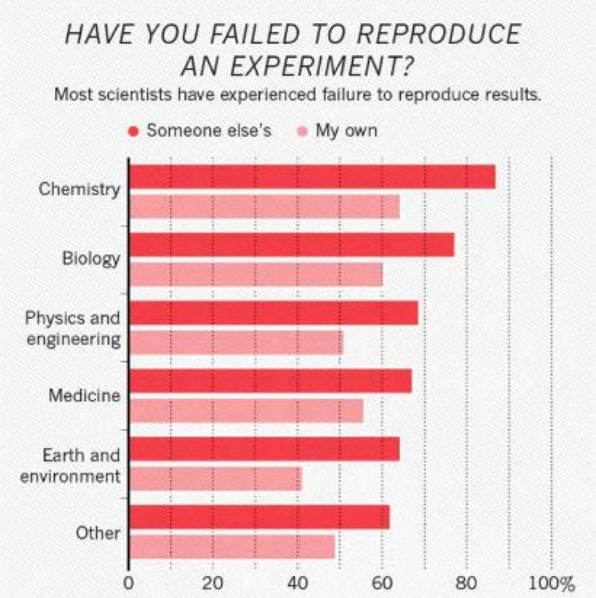
Beat Wolf

iCoSys HEIA-FR

Reproducibility

- Reproducibility of experiments is perceived as a given
- Yet 70% of researchers have trouble reproducing experiments
- In research and industry, reproducibility is key





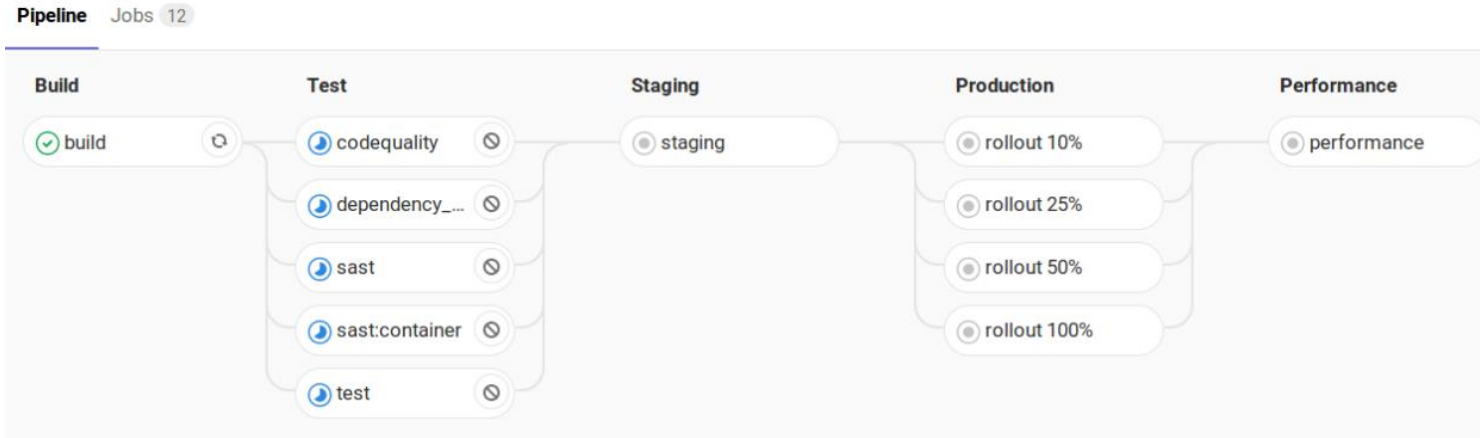
https://blog.ml.cmu.edu/2020/08/31/5-reproducibility/

What is reproducibility?

- There are different types of reproducibility
  - **Method**
    - The ability to rerun an experiment and get the same results
  - **Results**
    - The ability to apply a given method to a similar problem and get similar conclusions
- What is the key for doing that?
  - Knowing the exact version of the code used
  - How it was run (compilation, dependencies, etc.)
  - All parameters
  - The data

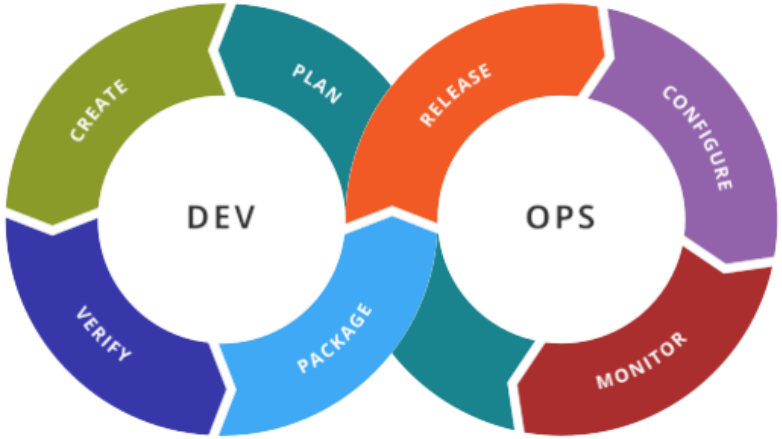
Classical programming

- In the world of classical programming many of those problems have been tackled
  - Usage **of standard algorithms** in modern IDEs
  - **Git** to version the code
  - **Tests** (unit, system, integration) to validate the system
  - Automation through **CI/CD** systems
    - Going from the raw code to the final deployed system fully automated



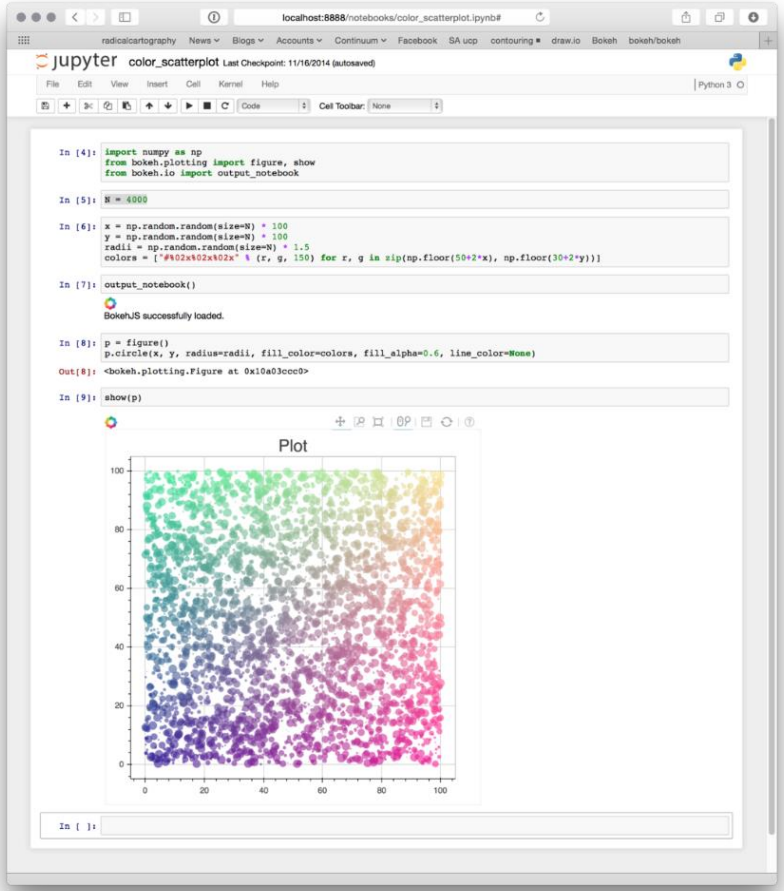
DevOps

- All this has been combined in the principle of DevOps, the Unification of **Dev**elopement and **Op**erations
  - Automate all steps to avoid issues like « but it runs on my computer»
- Largely based on automation such as:
  - Tests, builds (Docker, Jars etc.), deployment (kubernetes etc.), monitoring
- DevOps is not just automation and CI/CD pipelines, it is a philosophy



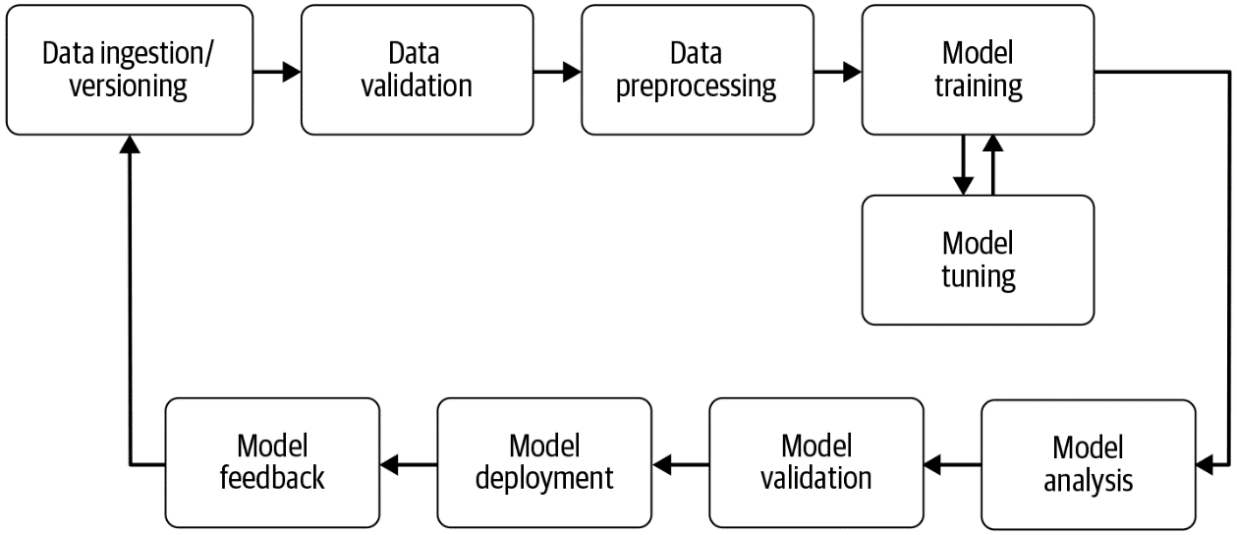
Machine learning

- Rapid iteration with Jupyter Notebooks
  - Usually with no versioning of changes
- Large quantities of data
  - No versioning either
- Manual training of models
  - On developer machine or cloud environment like google collab
- Difficult to know what code, parameters and data has been used to create a particular model
- DevOps principles are rarely applied



Steps in a ML pipeline

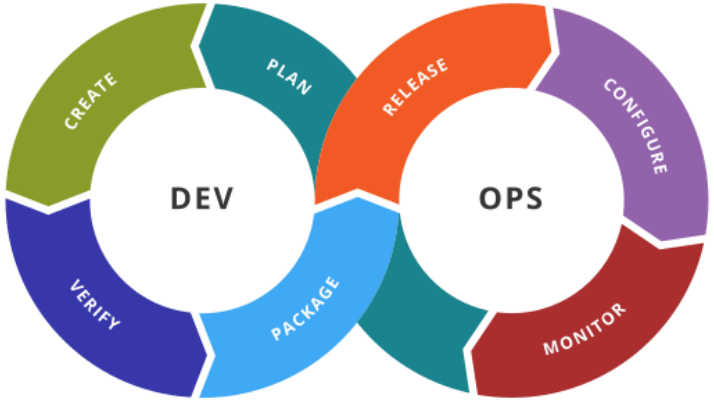
- Splitting the development of a ML model in phases makes it easier to version, automate and evaluate them.
  - Data ingestion/versioning
    - Storage, versioning
  - Data validation
    - Is the data correct? Does it satisfy my needs?
  - Data pre-processing
    - Convert, clean, complete
  - Model training /Model Tuning
    - Tuning, optimisation, etc.
  - Model analysis
    - Performance, validity, bias, etc.
  - Model deployment
    - Microservices, client, etc.
  - Model feedback
    - Drift, performance, etc.
- **And we start again**

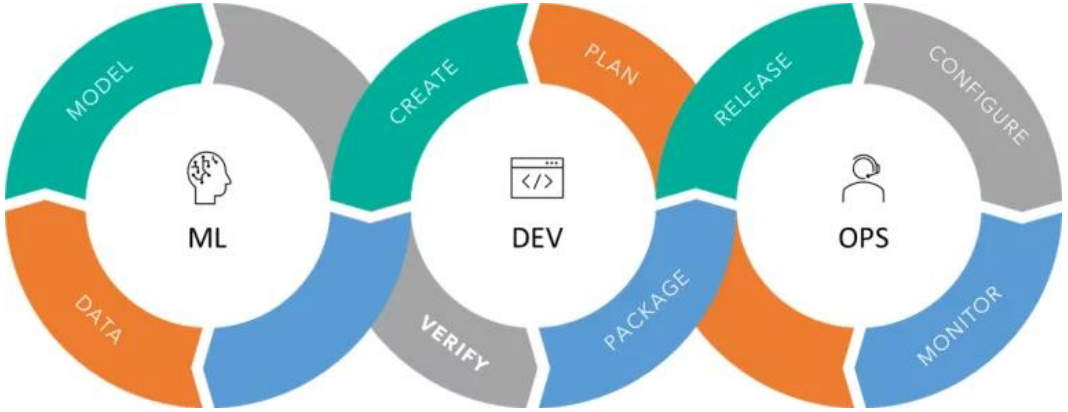


Building Machine Learning Pipelines

What is MLOps ?

- Integration of **ML** development into the Dev**Ops** process
- Largely based on DevOps technology
  - Completed with new tools if necessary
- Main things to add to DevOps:
  - **Logging** of experiments (documentation!)
  - **Versioning** of the data
  - **Scalability / reproducability**





https://techinnocens.com/mlops-fast-tracking-ethical-use/

MLOps technologies

- MLOps is still a very young concept (only a few years old)
- The technological landscape is quickly evolving

git, Git LFS, CML CI/CD for ML, neptune.ai, Apache Spark ML, Kubeflow, lakeFS, DVC, mlflow, CI/CD, kubernetes, aws

- And many more!
- Today we just look at a small collection

Jupyter Notebooks -> MLOps

- Going from Jupyter Notebooks to a more maintainable and reproducible environment can be incremental

- **Experiment logging**
  - Experiment logging frameworks
- **Data versioning**
  - Data versioning tools
- **Automation**
  - Creation of pipelines

Experiment logging/tracking

Experiment logging

- Classical approach is to document the experiments by hand
  - Different hyperparameters, pre-processing etc
- Quickly unmanageable and hard to reproduce

```
0000 p1 DRZ 3.11 - not super eval
mag_threshold 0.21
2L 300N BLSTM (BasicLSTM)
20D
sigmoid
AdamOptimizer
100 frames
dropout 1.0
zero input and label
log(x+1.0)
103300 training, 2000 CV
model:
weights20170224-005946_v10.1419
(p1, loss .1419, epoch 40 [task0])
MEAN IBM SDR GAIN:      2.324 -
with 0.15 thresh during cluster
STD IBM SDR GAIN:       2.276
MEAN IBM SDR GAIN:      2.110 -
with 0.32 threshold during cluster
STD IBM SDR GAIN:       2.254
```

```
0001 p2 DRZ 3.23
mag_threshold 0.12
2L 300N BLSTM_clean (LSTM & many
reworks) - note, this was the
massive model rewrite
20D
sigmoid
AdamOptimizer
100 frames
dropout 1.0
zero input and label
log(x+1.0)
10330 training, 2000 CV
model:
weights20170224-032054_v10.1418
(p2, loss .1418, epoch 40 [task0])
MEAN IBM SDR GAIN:      2.056 -
with 0.15 thresh during cluster
STD IBM SDR GAIN:       2.214
MEAN IBM SDR GAIN:      2.068 -
with 0.32 threshold during cluster
STD IBM SDR GAIN:       2.205
```

```
0002 p3 DRZ 3.66
mag_threshold 0.32
2L 300N BLSTM_clean (LSTM & many
reworks)
20D
sigmoid
RMSOptimizer(learning_rate=.01 *
0.5^int(epoch/50))
100 frames
dropout 1.0
zero input and label
log(x+1.0)
10330 training, 2000 CV
weights20170224-062901_v10.1478
(p3, loss .1478, epoch 47)
MEAN IBM SDR GAIN:      2.977 -
with 0.15 thresh during cluster
STD IBM SDR GAIN:       2.240
MEAN IBM SDR GAIN:  2.845 - with
0.32 and super_eval script
STD IBM SDR GAIN:   2.375
```

https://towardsdatascience.com/machine-learning-experiment-tracking-93b796e501b0

Technologies

- Many frameworks for experiment logging exist
  - https://clear.ml
  - https://neptune.ai
  - https://wandb.ai
  - https://www.comet.ml
  - https://www.tensorflow.org/tensorboard
  - https://github.com/jungomi/lavd
  - Etc.
- They all have similar features to log:
  - Metrics
  - Plots
  - Code
  - Data

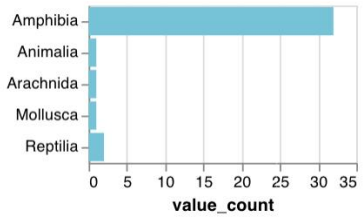
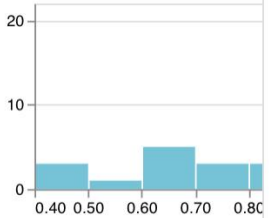
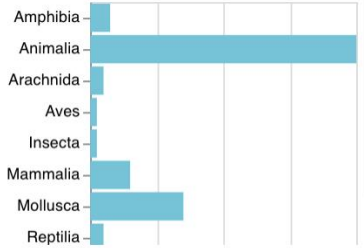
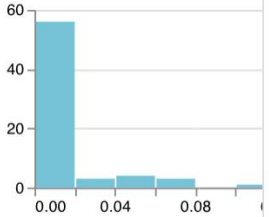
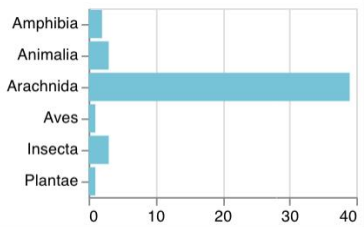
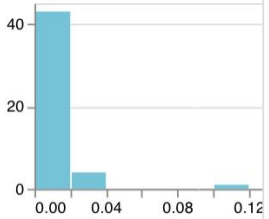
Weights and biases

- Modern experiment tracking allows to have a quick overview of our experiments
- Relevant information is tracked automatically
- Allows for easy comparisons between experiments

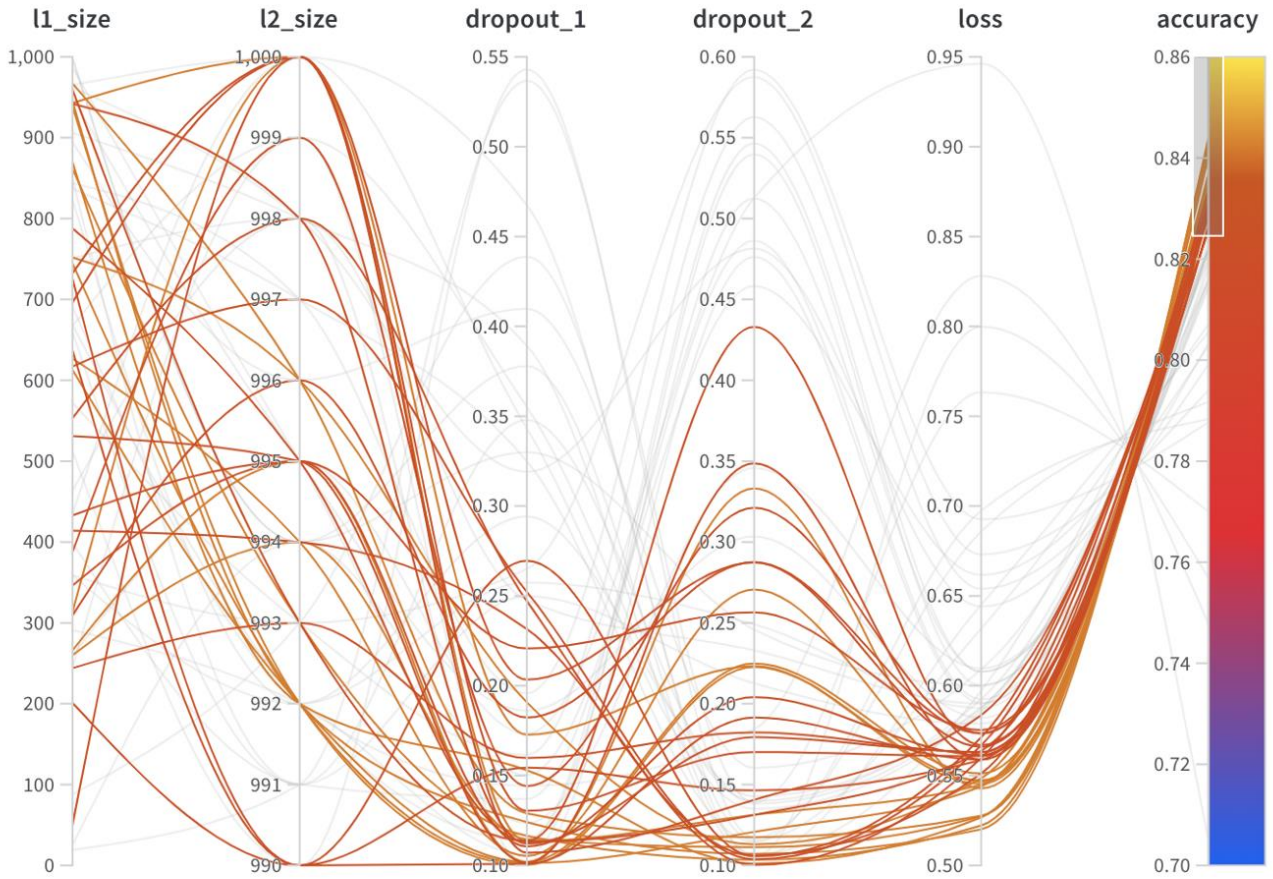
Name (384 visualized)	Tags	Created	Runtime	Sweep	encoder	num_train	num_valid	acc	car_acc	epoch	human_iou	iou	road_acc
worldly-totem-422	y	1mo ago	12m 54s	-	resnet34	682	97	0.8566	0.9169	4	NaN	0.7523	0.9175
jumping-voice-421	y	1mo ago	11m 59s	-	resnet34	725	92	0.8504	0.9381	4	NaN	0.7449	0.9141
logical-energy-420	y test_only	1mo ago	2m 14s	-	resnet34	66	10	0.626	0.0423	4	9.030e-8	0.4297	0.7683
laced-dust-419	y test_only	1mo ago	2m 4s	-	resnet18	61	15	0.5968	0.3018	4	8.677e-8	0.4701	0.75
whole-music-418	y test_only	1mo ago	1m 40s	-	resnet18	68	13	0.6139	0.4746	4	7.775e-8	0.4728	0.5818
grateful-glitter-417	y test_only	1mo ago	21s	-	resnet18	70	11	0.2367	0.00000…	0	9.091e-8	0.1209	0.9019
efficient-lake-416	y test_only	1mo ago	1m 42s	-	resnet18	76	10	0.6899	0.2821	4	NaN	0.4903	0.8558
clear-night-415	y test_only	1mo ago	1m 17s	-	resnet18	66	10	0.5403	0.06732	1	8.878e-8	0.3764	0.8544
glorious-night-414	y test_only	1mo ago	1m 33s	-	resnet18	68	7	0.7627	0.02521	3	7.359e-8	0.5818	0.873
smart-sponge-413	y test_only	1mo ago	1m 46s	-	resnet18	72	11	0.6517	0.1869	4	7.501e-8	0.4796	0.8491

Weights and biases

Filter | Reset & Automate Columns

Group by (guess) ↑	truth	score_Amphibia
Amphibia		
Animalia		
Arachnida		

Weights and biases



Weights and biases

```
import wandb
wandb.init(project="sunspots", entity="beatwolf", name="GradientBoostingRegressor")
wandb.config.learning_rate = 0.01 #Can also be used as a dict!

#TRAIN MODEL

wandb.log({f"accuracy" : accuracy, f"some_other_metric" : metric})

#Do some other analysis

plt.plot(my_values) #We can even log plots!
wandb.log({"pred_plot": plt})

wandb.log({"my_custom_plot_id" : wandb.plot.line(my_values, "x", "y",
           title="Custom Y vs X Line Plot")})

wandb.finish()
```

Monitoring and tracking of experiments

- With very little code, the frameworks allow us to track our experiments
- The different frameworks allow for easy integration in libraries like
  - Scikit-learn
  - PyTorch
  - Tensorflow
- The frameworks can do much more than just tracking metrics
  - Storing/versioning data and models
  - Interactive plots
  - Etc.
- Very useful when working alone, crucial when working as a team

Data versioning

Data versioning

- Git is not adapted to store binary data
  - Even less for large amounts
- Specialized solutions exist, such as
  - Etc.
- Most tools work with the same principle
  - Store small meta data of data files in git repository
  - Store actual data files (in different version) in external storage (like S3)

Data versioning with DVC

- DVC (**D**ata **V**ersion **C**ontrol) is a simple python tool to handle data versioning
  - `pip install dvc`
- DVC stores data files as .dvc files in your git repository
- Inside a git repository, create a DVC project
  - `dvc init`
- And then add files or folders to dvc
  - `dvc add MY_FILE_OR_FOLDER`
- A corresponding MY_FILE_OR_FOLDER.dvc file will be created

```
outs:
- md5: f3bc5178025fa7475da562eaf551e807.dir
  size: 1497247206
  nfiles: 22
  path: raw
```

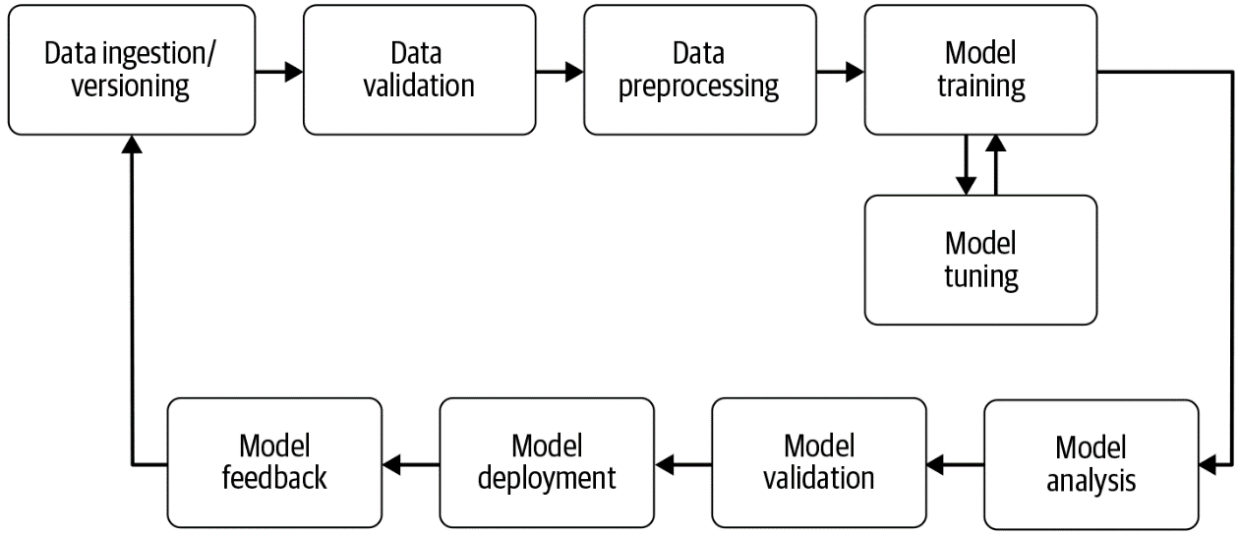
Data versioning with DVC

- To actually store the data outside your localhost, DVC requires a remote
  - dvc remote add -d myremote s3://mybucket/path
- Once all this is done, the workflow is very similar to git
  - dvc commit
  - dvc push
  - dvc pull
  - dvc update

Pipelines

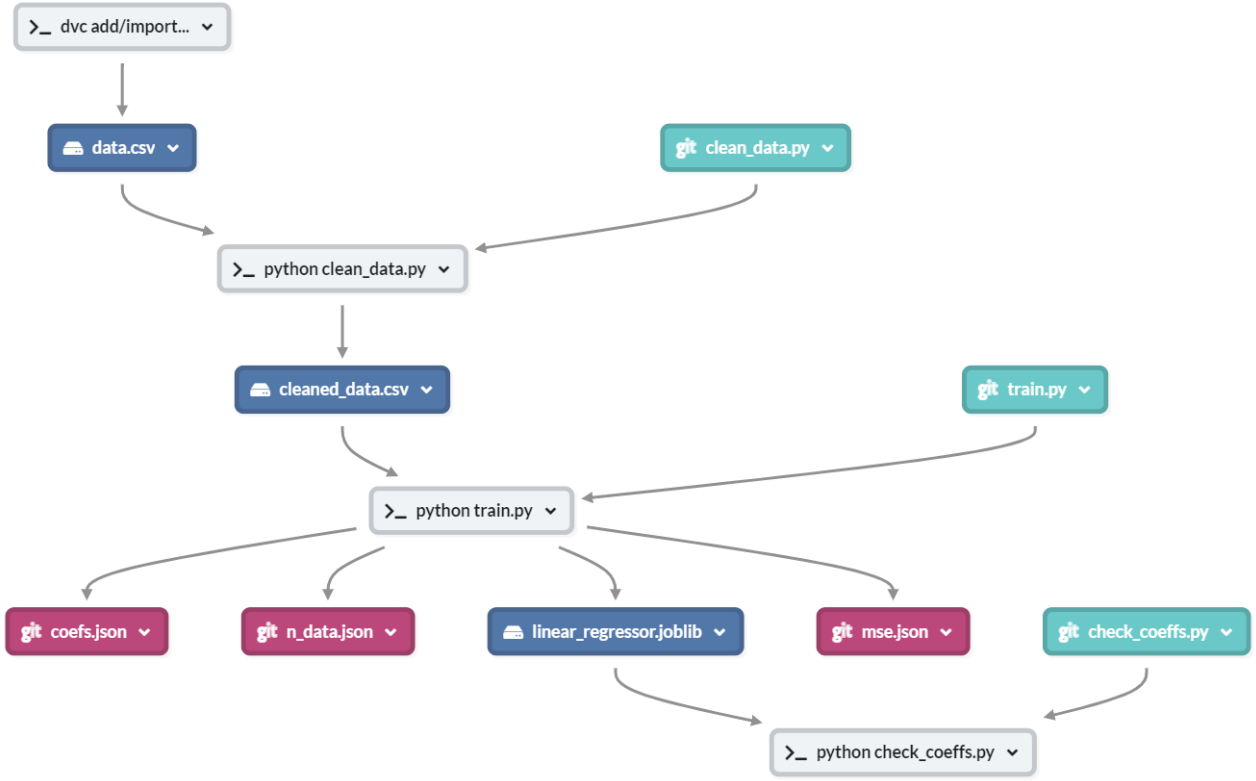
Automation and pipelines

- Last piece in the puzzle, pipelines
- Ideally we can split our process into individual steps that can be chained
- For this we require chainable tools, not Jupyter notebooks
- Examples:
  - Data validation reads a data file and outputs a Markdown report
  - The data pre-processing step reads a json file and outputs a csv file ready for the model training



Pipeline & DAG (Directed Acyclic Graph)

- The different parts of the pipeline can be seen as a Directed Acyclic Graph (DAG)
- Each step has its own inputs and outputs
  - Which can be shared / reused
- Once the pipeline is defined, we can version it
  - We know exactly what version created which model
- Different tools exist to define such pipelines and execute them
  - Locally or in the cloud



https://dagshub.com/

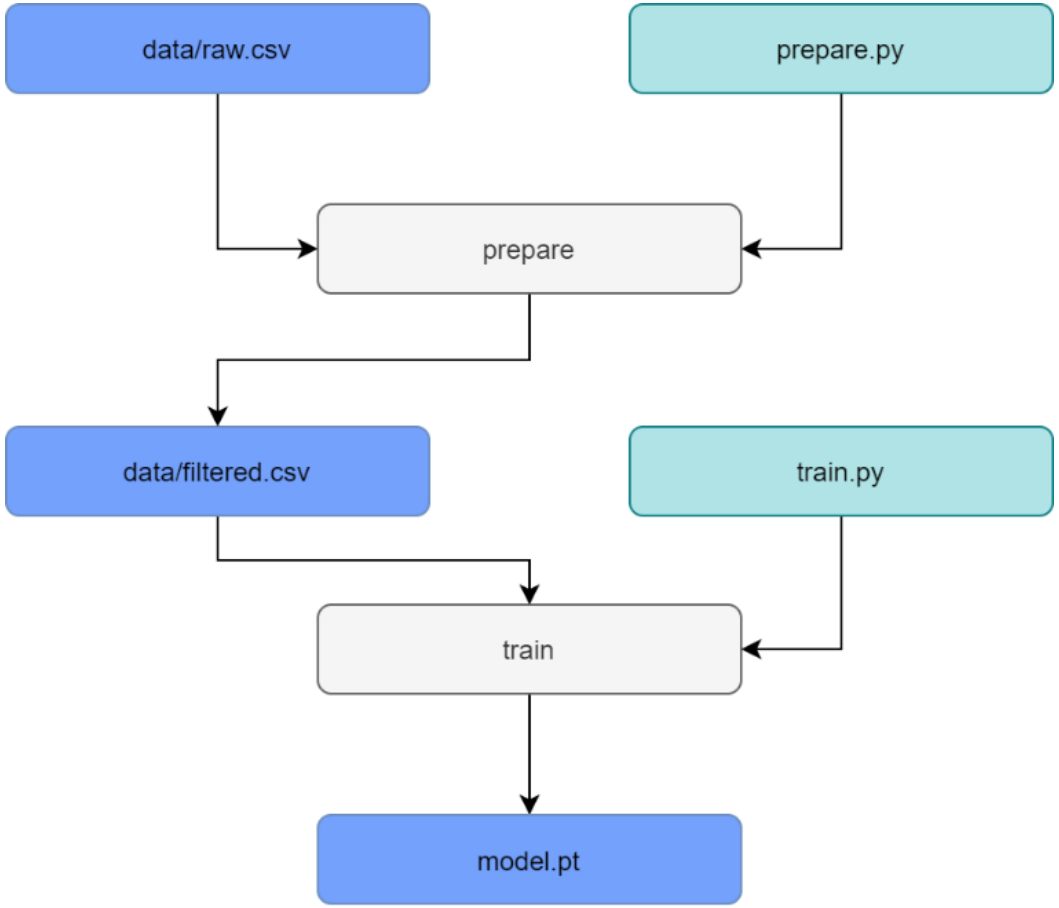
DVC example

- We can define pipelines with DVC
  - YAML file called dvc.yaml
- Once defined, we can reproduce it
  - dvc repro
- Every step defines a command, and all the inputs and outputs
  - The inputs and outputs will be hashed to know if they changed since the last run
- By default, all outputs are versioned by DVC

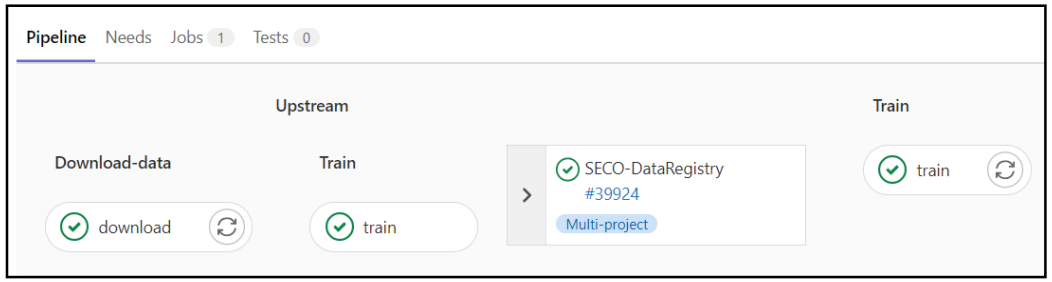
```yaml
stages:
  prepare:
    cmd: python prepare.py data/raw.csv flt.csv
    deps:
      - prepare.py
      - data/raw.csv
    outs:
      - data/flt.csv
  train:
    cmd: python train.py data/flt.csv mdl.pt
    deps:
      - train.py
      - data/flt.csv
    outs:
      - mdl.pt
```

DVC example

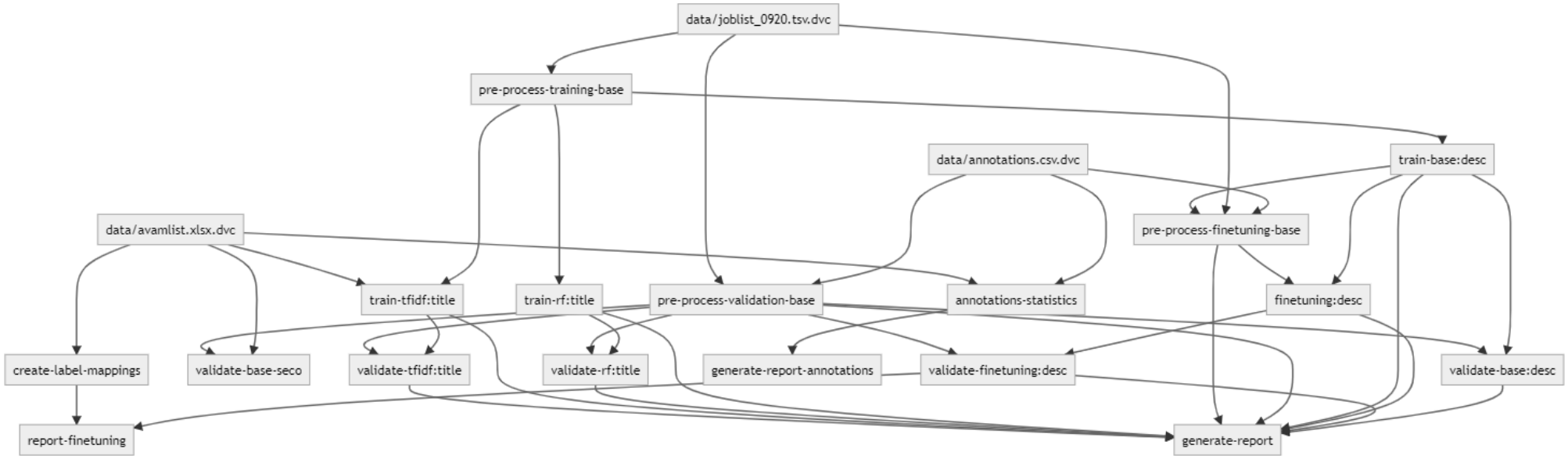
```
stages:
  prepare:
    cmd: python prepare.py data/raw.csv flt.csv
    deps:
      - prepare.py
      - data/raw.csv
    outs:
      - data/flt.csv
  train:
    cmd: python train.py data/flt.csv mdl.pt
    deps:
      - train.py
      - data/flt.csv
    outs:
      - mdl.pt
```



Example pipeline



- Daily retraining of model based on new data
  - Automatic training and comparison of different model with report generation



Example pipeline

Annotation statistics

Matching between experts and others

Majority voting

Path	AVAM	CHISCO_3	CHISCO_5
metrics/annotations/metrics_matching_majority.json	0.70876	0.83096	0.77393

'At least one' voting

Path	AVAM	CHISCO_3	CHISCO_5
metrics/annotations/metrics_matching_at_least_one.json	0.8391	0.93075	0.89206

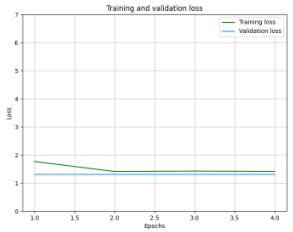
Matching between experts

Path	AVAM	CHISCO_3	CHISCO_5
metrics/annotations/metrics_experts.json	0.52488	0.67662	0.61194

Matching between others

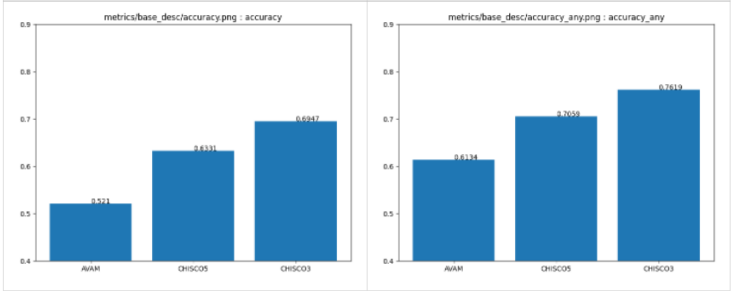
Path	AVAM	CHISCO_3	CHISCO_5
metrics/annotations/metrics_others.json	0.60478	0.72482	0.65139

Finetuning training



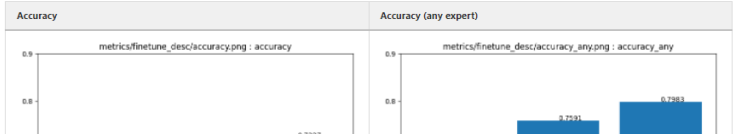
Path	accuracy	balanced_accuracy	f1_macro	f1_micro	f1_weighted	test_loss
models/bert_finetuned_desc/metrics.json	0.6452	0.0486	0.0482	0.6452	0.6447	0.8672

Base validation



Path	AVAM.accuracy	AVAM.accuracy_any	AVAM.balanced_accuracy	AVAM.f1_macro	AVAM.f1_micro	AVAM.f1_weighted	C
metrics/base_desc/metrics.json	0.521	0.6134	0.5555	0.3896	0.521	0.5416	0.

Finetuning validation



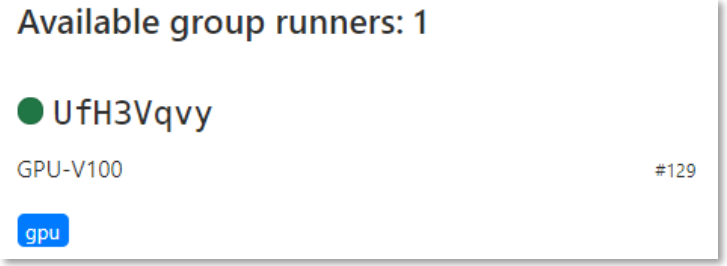
Infrastructure

Infrastructure

- Most things can be done locally (such as DVC)
- Quickly you are going to need an appropriate infrastructure
  - Storage, resource problems, sharing etc.
- You need versioning servers (code and data)
  - Git (Gitlab / Github) and DVC (AWS S3 / Minio etc)
- The ability to execute the pipeline in an appropriate infrastructure
  - Gitlab CI, in an infrastructure such as Kubernetes
- And the final deployment
  - Cloud, Kubernetes etc. depending on your needs

CI/CD integration for MLOps

- Even if it is possible to execute the DVC pipelines locally, it is preferred to centralize their execution
  - For the same raison that in classical development the final binaries are compiled in a CI/CD environment and not locally
- Requires a dedicated infrastructure, such as kubernetes
  - Adding individual computers with GPUs as CI/CD runners is very easy with Gitlab or Github




```
train:
    stage: train
    tags:
        - gpu
```

Integration into CI/CD

- Having an automated pipeline allows for automatic validation of new models
- CML (Continous Machine Learning), a DVC subproject, gives you gitlab/github integration
- For example, automatically compare new model in merge request with current main branch

Elle O'Brien @andronovhopf commented on commit fac0c8ce 8 months ago

Maintainer

Style transfer

Workspace vs. Master

Training metrics

Path	Param	Old	New
params.yaml	style_weight	100.0	50.0

GPU info

Metric	Value
Iterations	2
Run time (s)	779.615
Final loss	5.16476e+10

Beat Wolf @beat.wolf commented on commit 7265a45a 2 weeks ago

Metrics

Base traning

Path	Metric	Old	New	Change
models/bert_base_title/metrics.json	accuracy	0.8511	0.8431	-0.008
models/bert_base_title/metrics.json	balanced_accuracy	0.379	0.3589	-0.0201
models/bert_base_title/metrics.json	f1_macro	0.3756	0.3547	-0.0209
models/bert_base_title/metrics.json	f1_micro	0.8511	0.8431	-0.008
models/bert_base_title/metrics.json	f1_weighted	0.8379	0.8277	-0.0102

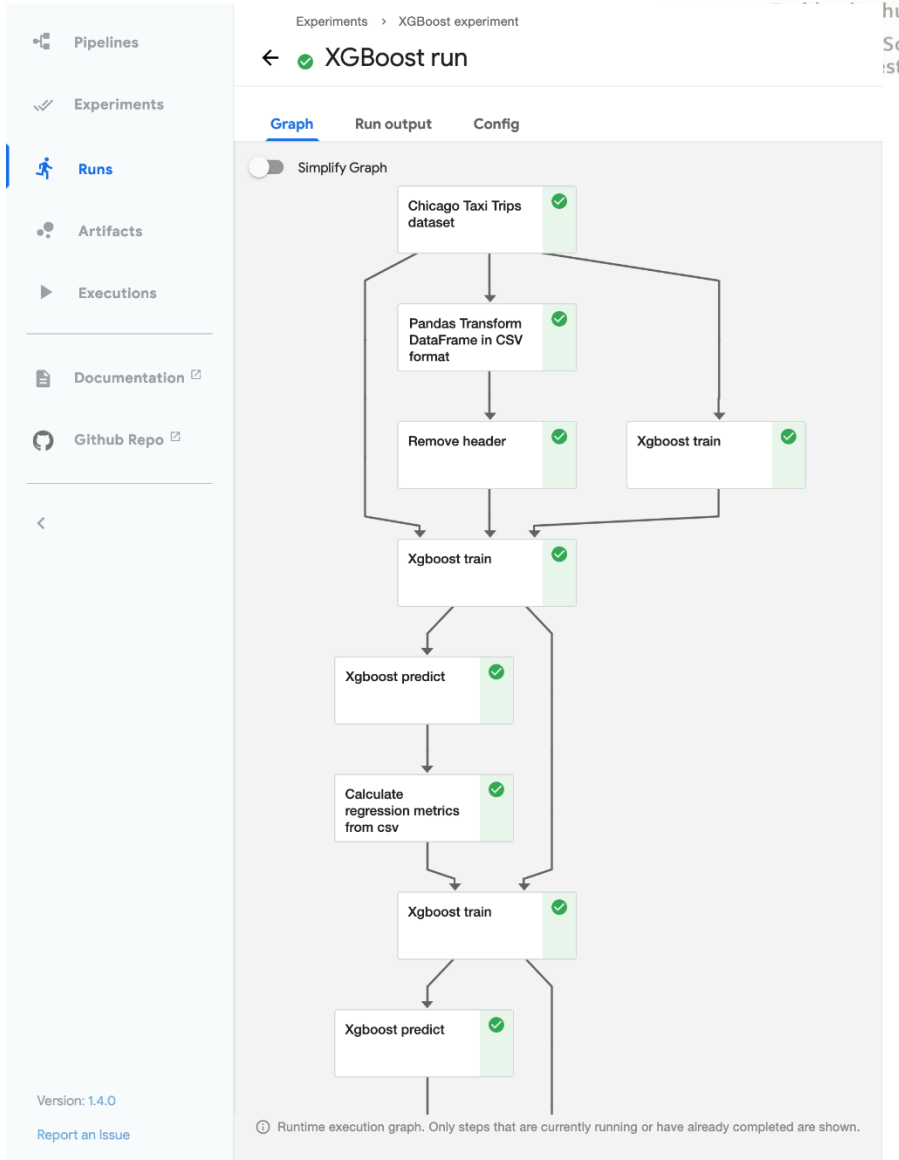
CML CI/CD for ML

Distributed MLOps pipelines

- Often the handling of MLOps pipelines is difficult, even with DVC and similar tools
  - How to correctly connect the different steps in a distributed environment?
  - How to distribute the different steps?
- Frameworks exist to simplify this task
- Kubeflow being one example
  - There are others, such as MLFlow, MLRun etc.

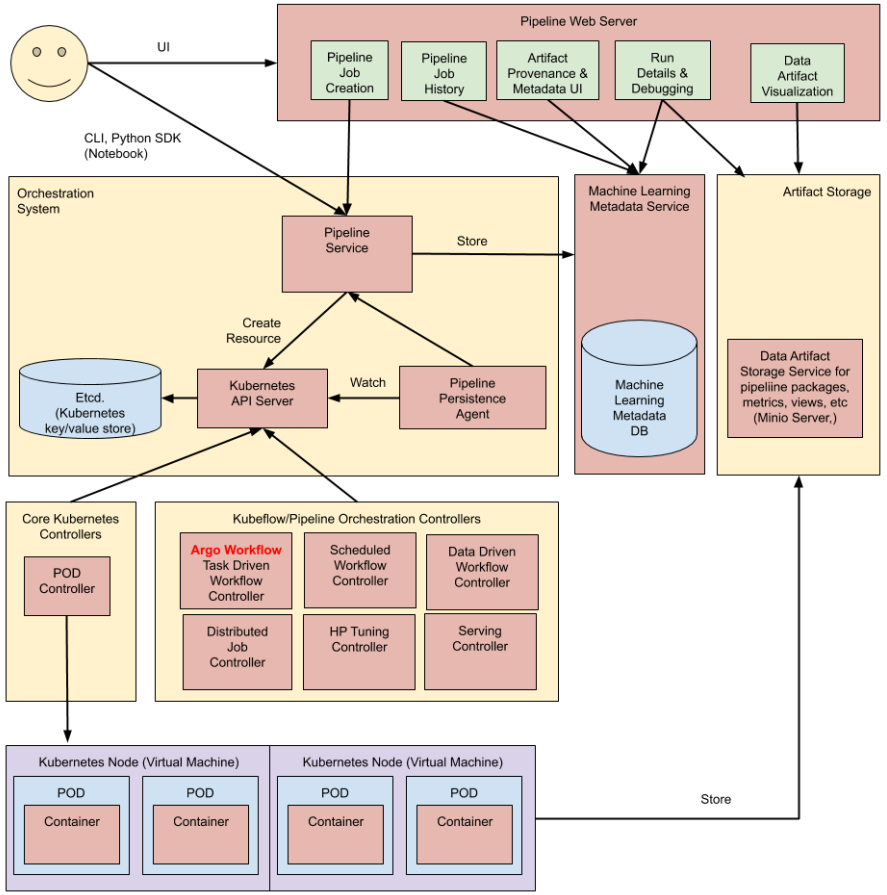
Kubeflow

- Kubeflow is an open source project, originating from google
- Kubeflow allows you to define pipelines using python code
  - Similar to DVC yaml files
- Every step of the pipeline is automatically containerized and distributed
- Requires a dedicated kubernetes cluster
  - With the appropriate resources, such as GPUs



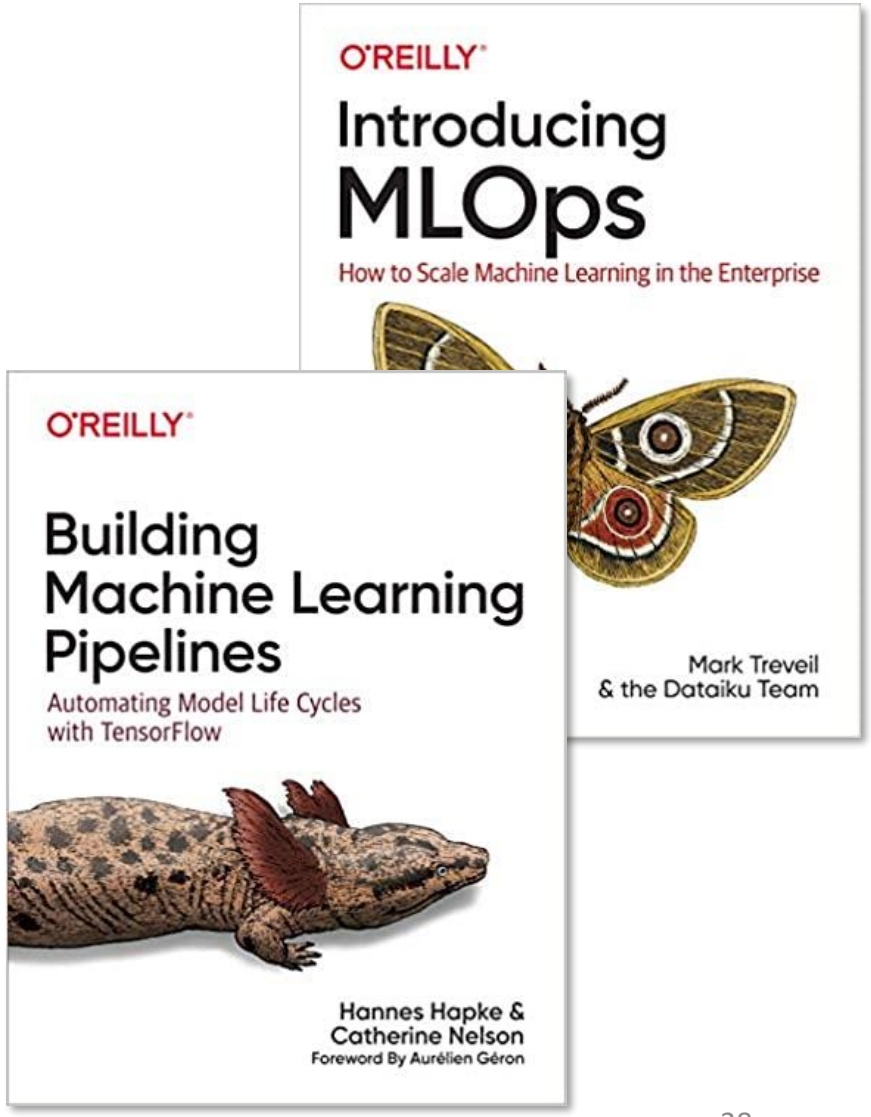
Kubeflow : Architecture

- Kubeflow will automatically distribute and coordinate your pipeline on a Kubernetes cluster
- Also includes tracking of the experiments and a centralized dashboard



MLOps conclusion

- With simple tools you can progressively go from a Jupyter Notebook to a professional pipeline
  - Adding experiment tracking
  - Adding data versioning
  - Splitting Jupyter notebook into individual blocks to create a pipeline
    - Can be done progressively, for example by starting with the data pre-processing
  - You do not required a special infrastructure, but it can help
- As much as DevOps has become a must have in traditional programming, MLOps will be in the future for ML projects
  - Model performance being dependant on which developer trains the model is not acceptable
- Try to think of you models and data as you think of code
  - Automate and version as much as you can

Resources

- Books
  - *Introducing MLOps*
  - *Building Machine Learning Pipelines*
    - Check your university library!
- Videos
  - Check out the youtube channels of
    - Google
    - Amazon
    - DVC/CML
    - Kubeflow
    - Databricks
- Other resources
  - All resources concerning DevOps, DataOps and MLOps